Data Structures and Object-Oriented Programming I

This core computer science course covers the internal workings of algorithms and data structures, from mathematical principles to implementation in C++. Topics include development of algorithms; algorithm analysis; object■oriented programming; abstract data types including stacks, queues, and trees; discrete math including solving recurrence relations, number representation, and predicate logic; programming concepts including memory management and pointers; modern software engineering practices.

This is a fast-paced and demanding course that assumes no prior exposure to C++. By the end of this quarter, you will be a confident C++ programmer and will be comfortable with the basics of object-oriented design and programming. You will understand how to analyze a problem and design a solution, recognizing when existing techniques and software are reusable. You will understand the tradeoffs among memory, running time, and implementation time associated with different data structures and algorithms.

The subject matter is highly technical so plan to put in considerable time and effort master the material — you are expected to perform at a graduate level. Expect to spend an average of more than 15 hours a week for this course; some of you may spend more, some less time.

Course Objectives

The goals of this course are for you to be able to do the following:

- Apply pictorial representations to explain the implementations of data structures and their operations.

- Describe and implement algorithms associated with data structures.

- Develop recursive algorithms to solve novel problems.

- Explain the operation of fundamental algorithms, such as for searching and sorting.

- Design and produce code using object-oriented programming.

- Relate mathematical concepts, such as recurrence relations, number systems, and logic, to software development pragmatics.

Instructor

Michael Stiber <stiber@uw.edu>.

Please see Signing Up for Office Hours for how to schedule meetings with me or our TA.

If you have a time-critical communication, you can message Prof. Stiber using Microsoft Teams using my UW email address.

TA

Carl Mofjeld <cmofjeld@uw.edu>.

Textbooks

Frank M. Carrano and Timothy Henry, *Data Abstraction & Problem Solving with C++: Walls and Mirrors*, seventh edition, Pearson, 2017.

Charles A. Cusack and David A. Santos, *An Active Introduction to Discrete Mathematics and Algorithms*, version 2.6.4, May 24, 2019. Available as a free online PDF here. (Local cached copy is here.)

Suggested Readings

Bruce Eckel, *Thinking in C++*, Second Edition, vols. 1 & 2, Prentice Hall, 2000 and 2003. A good “from scratch” introduction to the language. These have historically been

available for free in electronic form on-line. Since the official links for them seems to be broken as of this writing, and I have not found anything requesting others take down any locally cached copies, you can download volume 1 and volume 2 from this Canvas course site.

Bjarne Stroustrup, *A Tour of C++*, Second Edition, Addison-Wesley, 2018. An accelerated intro to C++ for those already familiar with programming.

Bjarne Stroustrup, *The C++ Programming Language*, Fourth Edition, Addison-Wesley, 2013. The canonical reference, updated for C++11.

Nicolai M. Josuttis, *C++17: The Complete Guide*. No-nonsense book targeting folks familiar with C++ who need to update themselves to the latest standard. This is being published in an interesting way, with you deciding how much to pay for a draft and later, free, updates.

Harley Hahn, *Harley Hahn's Student Guide to UNIX*, 2nd edition, McGraw-Hill, 1996. If you're unfamiliar with Unix or Linux and want to learn more, get a book like this.

Nicolai M. Josuttis, *The C++ Standard Library: A Tutorial and Reference*, 2nd Edition, Addison Wesley Longman, 2012. Good introduction and useful reference for the C++11 standard library.

David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley, 2017. Tutorial and reference, updated for C++17.

Major Assignments

We will have both *written* assignments and *programming* assignments. Subsequent sections of this syllabus carefully spell out (in detail) both the procedures for program submission and the content of what you should submit. *Please read this syllabus and the All About Programming Assignments page carefully*. If there is anything you don't understand or are not sure about *ask me*. *I will assume that you have done so, and will mark off if what you submit does not match what is required*.

Tests

Test will be online using the ProctorU test proctoring system. Please see our Online Testing Using ProctorU page for more information and make sure you've got everything set well before the quarter.

Grading

Your course average is computed as: 30% homework + 25% final + 20% midterm + 10% designs + 5% peer design reviews + 5% discussions + 5% Quizzes

I don’t grade on a curve. I compute everyone’s quarter (percent) average based on the formula above, and then use my judgment to determine what averages correspond to different decimal ranges for the quarter. Some quarters’ assignments, etc. turn out harder, and so the averages are lower. Other quarters, averages are higher. This is adjusted for that at the end. Furthermore, I am well aware of the significance of assigning a grade below 2.7, in terms of impact on your career here at UWB. I can assure you that I examine in detail the performance in this course of each student before assigning a grade below 2.7.

What is the difference between this and grading on a curve? With the latter, the goal is to have X% ‘A’s, Y % ‘B’s, etc (or, their equivalent decimal grades). My way, I would be happy to give out all 4.0s (if they were earned). FYI, in a “typical” quarter, at 2.7 would correspond to around a 70-75% average, a 3.0 to around 80%, and around 90% or more would be 3.5 or above. You may use this as a rough guide; however, if you really want to know how you’re doing, please talk with me. *I reserve the right to adjust these scores to reflect the specifics of assignments, test questions, etc. for each quarter.*

Problems

If you have problems with anything in the course, please use office hours or make an appointment to speak with me at some other time. I want to make you a success in this course. If you have trouble with the assignments, see me before they are due. If you fall behind, it will be difficult to catch up. I will not give out grades of “incomplete” except in extreme circumstances.

Etiquette, Etc. (the "rules")

- Read this entire detailed syllabus, the All About Programming Assignments page, and the other class documents listed under the "Useful Links" link to the left. *Every word.* Failure to follow instructions (for example, neglecting coding or documentation standards, trying to turn an assignment in the wrong way or after a deadline) will adversely impact your grade.

- Work through all of the materials in a module and write down questions you have as they occur to you. Play with code while you're doing this as appropriate. You will find that you are able to answer many of your questions yourself as you learn. However, it's important to remember that *questions are good* — they are gateways to greater understanding. Don't hesitate to use the topical discussion forums to air your questions and *contribute your findings* in response to others' questions. We learn as a community. Part of your grade is based on your contribution to the community.

- I will not try force you to use any particular development environment. However, your programs *must* compile with g++ and run on the CSS Linux machines. Please be advised that, if you use any other environment, it is possible that you will spend considerable extra time "porting" your code to the class compiler and computing environment. It is even possible that you will never get your program working. It is a worthwhile investment of your time for you to learn to use Unix and g++. If you use another development environment, then you assume all responsibility for getting your code running on the class computing platform; we will *not* make exceptions and test programs otherwise.

- Similarly, there are certain class standards for documentation, including UML diagrams. If the tool you use does not produce diagrams that look like the class requirements, please find another tool. High-quality scans of neat hand drawings are perfectly acceptable.

- You are responsible for making back-up copies of your work. Disk crashes, etc. are *not* acceptable reasons for extensions of assignment due dates. Note that your Linux home directories are professionally backed up, as are Google Drive, Microsoft OneDrive, etc.

- Assignments are due when specified. Barring illness or similar extenuating circumstances, please do not attempt to submit amendments, bug fixes, or forgotten material after the fact.

- Please do not post your work to publicly accessible repositories. Use a private repo if you want to create a portfolio that you can grant access to.

- Do not seek out programming assignment solution code on the web (I will generally try to make our assignments so idiosyncratic that such code is unlikely to exist, anyway). You need to submit your own work.

- Please use your UW email (*netid*@uw.edu) for direct email communications with me (i.e., make sure that that shows as your return address) or use the built-in

Canvas messaging capability (via the Inbox). This is the only way that I can authenticate that you are who you say you are. I will endeavor to only email you at that address. I do this because that address is the only one guaranteed to be connected to you, based on information in the student database.

- I expect you to treat fellow students, and other UWB community members, with the utmost respect and consideration. You will be more successful, and happier, as a supportive and considerate member of the CSS graduate certificate community.

- You are expected to provide original work based on your own effort for this course. You will receive a zero for any coursework for which you are discovered cheating or plagiarizing. You will be referred to the University for further action. It is your responsibility to know and uphold the Student Conduct Code for the University of Washington.